ENGINEERING ASSESSMENT & MIGRATION PLAN

Relay
Lead Management Platform

Task B — Technical Assessment, Phased Migration, Refactor Study, and Engineering Standards

Subject	Production readiness of the existing Next.js + MongoDB codebase
Scope	Incremental improvement of the current architecture — no rewrite
Author	Engineer who designed and built Task A
Audience	Engineering leadership & reviewing panel
Status	Draft for review

Executive summary

The platform is well-structured: a strict route → service → repository → model seam, authorization scoping enforced at the data layer, a single error envelope, Zod validation at every boundary, and a green type/lint/test baseline. It is a solid foundation, not a finished product. This report identifies the highest-leverage gaps — non-atomic audit writes, an untested authorization invariant, no rate limiting on the one unauthenticated write path, and the absence of CI and observability — and lays out a strictly additive, zero-downtime path to production hardening that preserves the existing architecture.

Contents

0 · Context & method

This assessment is written from the inside. I built the current system, so the goal here is not to critique an unfamiliar codebase but to be honest about where I stopped and what production would demand next. Every recommendation maps onto structures that already exist, and the guiding constraint is **incremental change behind the existing seams** — no speculative rewrite.

What the codebase already gets right, and which I intend to preserve:

- **A real layered seam.** Routes and Server Components call services; services call repositories; repositories own Mongoose. Business rules never leak into route handlers.
- **Authorization at the data layer.** `leadRepository` merges a scope on every read and write (`deletedAt: null` always, `assignedTo = self` for members), so a member can never reach another rep's lead even if upstream guards are bypassed.
- **One error contract.** `withErrorHandler` maps ZodError → 400 and typed `AppError` subclasses → their status, behind a single `{ data }` / `{ error }` envelope.
- **Edge-safe auth split.** `lib/auth/config.ts` (no Node deps) powers middleware; `auth.ts` adds the Credentials provider on the Node runtime.
- **Green baseline.** `tsc --noEmit`, ESLint, and Jest (11 tests) all pass; Playwright smoke and a Netlify adapter are wired.

The layered request path that every recommendation respects:

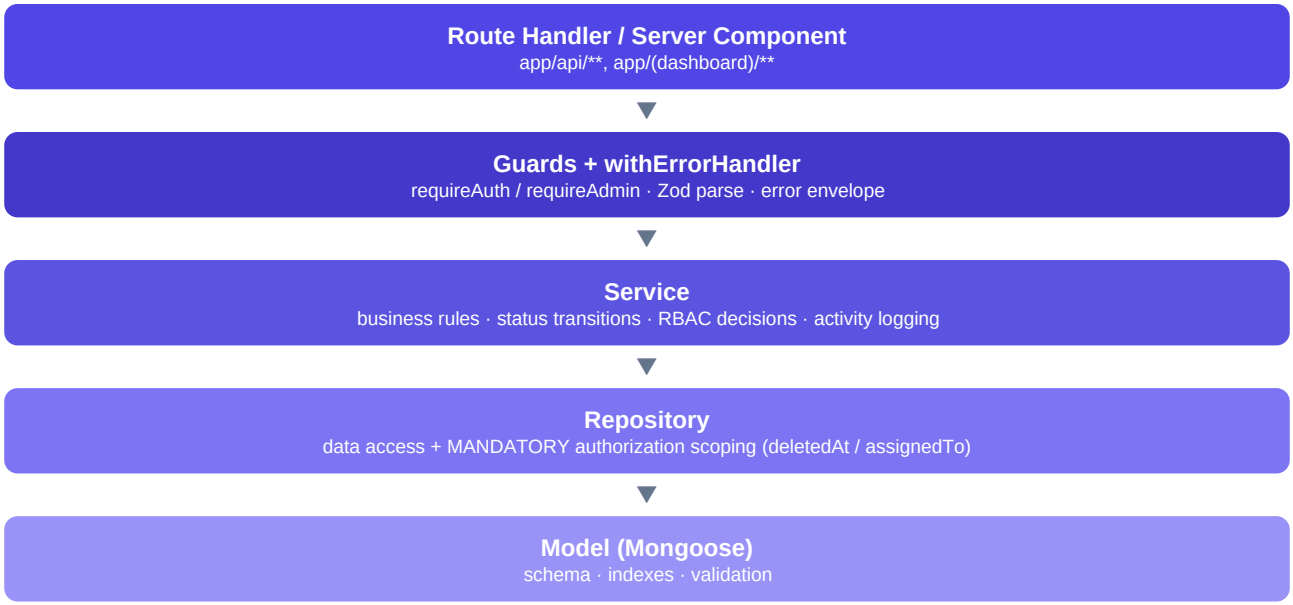

Figure 1 — Current request lifecycle. Improvements are inserted layer-by-layer, never around the seam.

1 · Assessment: what to fix first

Issues are ordered by the cost of leaving them unresolved, not by effort. The first three are **P0** because each is either a correctness or a security invariant that is currently unguarded, yet each is cheap to close precisely because the architecture already centralises the relevant code.

Issue (grounded in the codebase)	Priority	Risk if left unresolved	Engineering reasoning
Audit writes are not atomic. In leadService, the lead write and the ActivityLog write (and the multiple logs in updateLead) are separate awaits with no transaction.	P0	A failure between writes leaves a lead with no history, or history with no state change. The activity timeline and dashboard 'recent activity' silently drift from reality; an audit trail you cannot trust is worse than none.	Both writes already live in one service method — the natural place for a Mongo session/transaction. Blast radius is one file per operation.
The core tenant-isolation invariant is untested. Repository scoping (member sees only own leads) has zero automated coverage, despite mongodb-memory-server already being a dev dependency.	P0	The single most security-critical rule can be broken by an innocent refactor and no test will fail. This is how data-leak incidents ship.	Add integration tests that seed two members and assert cross-access returns nothing / 404. The tooling is already installed; this is pure upside.
No rate limiting on the unauthenticated capture endpoint (POST /api/leads) or on the auth route.	P0	The one public write is open to spam and lead-poisoning; the login route is open to credential stuffing. Both inflate Atlas cost and pollute the pipeline.	A token-bucket check in middleware (or Upstash Ratelimit) keyed by IP. Additive, no schema change, sits in front of the existing handler.
Search uses an unindexed regex $or across name/email/company; listing uses offset pagination.	P1	Both are fine at seed scale and degrade predictably: full collection scans and expensive deep skips as leads grow into the tens of thousands, raising latency and Atlas CPU.	A text index (already declared on the model) or Atlas Search covers the query; keyset pagination removes deep-skip cost. Repository-local change; API contract unaffected.
No observability. Errors are only console.error inside toErrorResponse; there is no structured logging, error tracking, or metrics.	P1	Production is effectively blind. Mean-time-to-detect and to-resolve are unbounded because there is no signal when a route starts failing.	withErrorHandler is a single chokepoint — wire pino structured logs and Sentry there once and every route inherits it.
No CI. Quality gates (typecheck, lint, test) run only locally via Husky/lint-staged.	P1	Nothing prevents a broken or untested change from reaching main; the green baseline is unenforced and will rot.	A GitHub Actions workflow running the same three commands on every PR. High leverage, ~30 minutes to set up.
Client mutations call router.refresh() after every write, re-rendering the whole server subtree.	P2	Correct today, but as pages grow it re-runs unrelated queries on each mutation, adding DB load and latency.	Move to targeted revalidateTag or TanStack cache updates. Behavioural parity, better efficiency.
No API contract artifact; client fetch helpers hand-type responses. Soft-delete has no restore path.	P2	Client/server types can drift silently; captured deletedAt/deletedBy are unusable, so admins cannot undo a delete.	Derive an OpenAPI doc / shared types from the existing Zod schemas; expose a restore endpoint reusing the scoped repository.

What I would fix first, and why in this order

Day one is the P0 trio because each is a latent invariant failure, not a feature gap: make the lead+activity writes atomic (correctness of the audit trail), add the RBAC/scoping integration tests (protect tenant isolation during future change), and put rate limiting in front of the two internet-facing write paths (abuse and cost). None of the three requires touching the architecture — they harden code that already exists at a single, well-defined location.

2 · Phased migration plan

The plan is deliberately staged so that value lands continuously and nothing is a big-bang. Each phase is **additive behind the existing service/repository seam**: we change one layer at a time, keep the public API envelope stable, and deploy through Netlify's atomic, instantly-revertible releases.

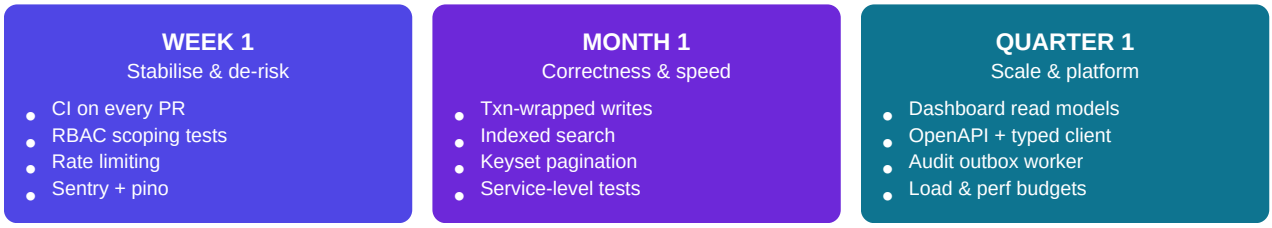

Figure 2 — Three horizons. Later work depends only on earlier hardening, never on a rewrite.

Week 1 — Stabilise and de-risk (no new features)

Goal: make the existing behaviour safe to change and observable in production.

- **CI pipeline.** GitHub Actions on every PR: `typecheck` → `lint` → `test`, required to merge. Codifies the current green baseline.
- **RBAC / scoping integration tests** on `mongodb-memory-server`: two members, cross-access denied; status-transition 409s; admin-only delete. Locks the P0 security invariant.
- **Rate limiting** in middleware for `POST /api/leads` (unauth) and `/api/auth`. IP-keyed token bucket.
- **Observability seam.** pino structured logs + Sentry wired once into `withErrorHandler`; a request-id added per response.
- **Secrets hygiene.** confirm AUTH_SECRET/URI are env-only, add dependency and secret scanning to CI.

> **Zero-downtime note (Week 1)**
> Every item is purely additive: new workflow files, new tests, a middleware guard, and logging inside an existing function. No schema change, no data migration, no API change — nothing that could take the app down.

Month 1 — Correctness and performance

Goal: close the P0 atomicity gap and the P1 performance cliffs while the system is small enough to change cheaply.

- **Transactional writes.** Wrap the lead mutation + activity log(s) in a Mongo session inside the service. Requires a replica set (Atlas is one by default). See the refactor in §3.
- **Indexed search.** Switch the regex $or to the model's text index (or Atlas Search) behind the same `listLeads` signature.
- **Keyset pagination** as an opt-in `?cursor=` alongside the current offset mode, so existing links keep working during the transition (expand/contract).
- **Granular revalidation.** Replace blanket `router.refresh()` with `revalidateTag('leads')` / detail-scoped tags.
- **Restore flow** for soft-deleted leads (endpoint + admin trash view) using the already-captured `deletedAt/deletedBy`.
- **Service-level test coverage** for transitions, assignment rules, and the last-admin guard.

Zero-downtime note (Month 1)
Search and pagination changes are repository-internal; the API response shape is unchanged. Keyset pagination ships side-by-side with offset (expand), clients migrate, then offset is retired (contract). Transactions change how a write executes, not its contract.

Quarter 1 — Scale and platform

Goal: prepare for data growth and multiple engineers without changing the mental model.

- **Dashboard read models.** Precompute the aggregates the dashboard recomputes on every request (materialised counts / trend), refreshed by the activity stream. Removes the heaviest read path.
- **Audit outbox.** Persist activity via a transactional outbox consumed by a small worker — decouples the write path and guarantees delivery even under load.
- **API contract.** Generate OpenAPI + typed client from the existing Zod schemas; version under `/api/v1`.
- **Performance budgets & load tests** in CI (p95 latency, query counts) so regressions are caught before release.
- **Tenancy hardening** review if the product grows toward multi-org: promote the implicit ‘scope’ into an explicit org boundary at the repository layer.

Zero-downtime note (Quarter 1)
Read models and the outbox are shadow-built and backfilled before any read is switched to them (expand → backfill → switch → contract). API versioning is additive: `/api/v1` launches beside the unversioned routes, which become aliases.

3 · Before / after refactor study

A realistic anti-pattern for this project: a create-lead route written the ‘fast’ way — Mongoose in the handler, role logic inline, no validation, no scoping, a non-atomic activity write, and the raw document returned to the client. It would pass a happy-path demo and fail in production.

Before — everything in the route handler

```
// app/api/leads/route.ts (anti-pattern)
export async function POST(req: Request) {
const session = await auth();
const body = await req.json();

// role logic inline; members can forge assignedTo
const lead = await Lead.create({
name: body.name, email: body.email,
status: body.status ?? 'New',
assignedTo: body.assignedTo, // no scoping, no validation
});

// second write, not atomic with the first
await ActivityLog.create({ leadId: lead._id, type: 'LEAD_CREATED' });

return Response.json(lead); // leaks the raw Mongoose doc
}
```

Problems, in order of severity: (1) the lead and the activity log are two independent writes — a crash between them corrupts the audit trail; (2) no authorization scoping, so a member can assign a lead to anyone; (3) no Zod validation, so `status` and `assignedTo` are attacker-controlled; (4) business logic sits in the transport layer and cannot be reused or unit-tested; (5) the raw document crosses the client boundary, leaking internal shape and dates. This is exactly what the layered seam exists to prevent.

After — the project’s pattern, plus the Month-1 transaction upgrade

Thin route: authenticate, validate, delegate. Identical to the shipped handlers.

```
// app/api/leads/route.ts
export async function POST(req: NextRequest) {
return withErrorHandler(async () => {
const user = await getCurrentUser();
const json = await req.json().catch(() => ({}));
if (!user) { // public capture path
const data = leadCaptureSchema.parse(json);
return ok(await leadService.captureLead(data), 201);
}
const data = leadCreateSchema.parse(json); // privileged fields validated
const dto = await leadService.createLead(
{ id: user.id, role: user.role }, data);
return ok(dto, 201); // DTO, not a Mongoose doc
});
}
```

Service owns the rule and now makes the two writes atomic (the current gap):

```
// lib/services/leadService.ts
async createLead(actor: Actor, input: LeadCreateInput): Promise<LeadDTO> {
// members can only ever create leads assigned to themselves
const assignedTo = actor.role === 'MEMBER'
? new Types.ObjectId(actor.id)
: input.assignedTo ? new Types.ObjectId(input.assignedTo) : null;

const lead = await withTransaction(async (session) => {
const created = await leadRepository.create(
{ ...input, assignedTo, createdBy: actor.id }, session);
await activityRepository.log(
{ leadId: created._id, actorId: actor.id, type: 'LEAD_CREATED' }, session);
return created; // both writes commit or neither does
});
return toLeadDTO(lead); // single mapper at the boundary
}
```

The transaction helper is a small, reusable wrapper over the cached connection:

```
// lib/db/withTransaction.ts
export async function withTransaction<T>(
fn: (session: ClientSession) => Promise<T>): Promise<T> {
await connectToDatabase();
const session = await mongoose.startSession();
try {
return await session.withTransaction(() => fn(session));
} finally {
await session.endSession();
}
}
```

Improvement	Before	After
Atomicity	two independent writes	one transaction — audit trail can never drift
Authorization	client sets assignedTo	member forced to self in the service; repo re-scopes
Validation	raw body trusted	Zod parse at the edge; privileged fields server-controlled
Separation	logic in transport layer	route delegates; rule is reusable and unit-testable
Boundary	raw Mongoose doc returned	toLeadDTO — string ids, ISO dates, no internal shape
Errors	unhandled / ad-hoc	withErrorHandler envelope, one contract for all routes

Net effect: the same feature becomes correct under failure, safe under a hostile client, testable in isolation, and consistent with every other endpoint — without inventing anything outside the architecture already in place.

4 · Engineering standards proposal

These standards describe how the team keeps the codebase in the shape it is already in. They are written as the rules this project follows, so adoption means ‘keep doing this, now enforced by CI’ rather than a new regime.

Coding standards

- **TypeScript strict** stays on, including `noUncheckedIndexedAccess` and `noImplicitOverride`. No `any`; prefer `unknown` + narrowing.
- **The seam is law:** Mongoose types never cross a service boundary; every server→client value is a DTO from `lib/mappers.ts`. Repositories are the only place that imports models.
- **server-only** on service/repository modules; `import type` for type-only imports (enforced by ESLint).
- **One source of truth** for domain rules (enums, STATUS_TRANSITIONS) in `src/types`, shared by models, Zod, and services.
- Prettier + ESLint (`next/core-web-vitals`) formatting is not up for debate; it runs in pre-commit and CI.

Folder structure

The current structure is the standard. A new feature adds a vertical slice across the same layers rather than a new top-level concept:

```
src/
app/(group)/feature/page.tsx // server component, calls a service
app/api/feature/route.ts // thin handler: guard + Zod + service
components/features/feature/*.tsx // client islands only
lib/services/featureService.ts // business rules + logging
lib/repositories/featureRepository.ts // data access + scoping
lib/validations/feature.ts // Zod schemas
models/Feature.ts // schema + indexes
types/dto.ts // DTO + mapper entry
```

Testing standards

- **Pyramid.** Many fast unit tests (rules, validation, mappers), a focused band of repository/service integration tests on `mongodb-memory-server`, a thin Playwright layer for critical flows (auth, capture, RBAC).
- **Non-negotiable invariants must have tests:** repository scoping (member isolation), status transitions, admin-only delete, last-admin guard. These gate merges.
- Coverage target 80% on `lib/services` and `lib/repositories`; components tested by behaviour (RTL), not snapshots.
- Every bug fix ships with the regression test that would have caught it.

Git workflow

- **Trunk-based** with short-lived branches; `main` always releasable.
- **Conventional Commits** (`feat:`, `fix:`, `refactor:`) driving an automated changelog.
- PRs are small, single-purpose, and use a template (what/why/risk/rollback). Required checks: typecheck, lint, test. One review minimum.
- Squash-merge to keep a linear, readable history.

API standards

- Single envelope: `{ data }` / `{ data, meta }` for success, `{ error: { code, message, details? } }` for failure — already the norm via `ok()` and `withErrorHandler`.
- Zod validation at the edge of every handler; privileged fields are server-assigned, never trusted from the client.
- Version under `/api/v1`; changes are additive (expand/contract), and the capture endpoint is made idempotent with a client key to survive retries.
- Consistent pagination contract; typed client generated from the schemas so client and server cannot drift.

Security standards

- Authorization enforced at the repository layer, not by hiding UI; guards and middleware are defence in depth on top.
- Rate limiting on all unauthenticated and auth endpoints; bcrypt cost ≥ 12; `passwordHash` stays `select:false` with the constant-time dummy-compare on unknown emails.
- Secrets only in environment (Netlify env / a secrets manager); never in the repo. Least-privilege Atlas user; IP allow-listing.
- No PII in logs; dependency and secret scanning in CI; security headers via middleware.

CI/CD

- GitHub Actions: **install** → **typecheck** → **lint** → **unit/integration tests** → **build** on every PR; Playwright smoke on a preview deploy.
- Netlify deploy previews per PR; merge to `main` promotes an atomic, instantly-revertible release.
- Later: performance-budget and migration-safety checks as required gates.

Monitoring

- Error tracking (Sentry) and structured logs (pino) wired through the single `withErrorHandler` chokepoint, with a per-request id.
- Uptime + synthetic checks on the capture and login paths; Atlas performance metrics and slow-query alerts; product KPIs (capture rate, conversion) on the dashboard read model.
- Alerting on error-rate and p95 latency SLOs, routed to the on-call channel.

Documentation

- The README stays the entry point (setup, scripts, API table, RBAC matrix).
- **ADRs** for load-bearing decisions (JWT sessions, repository scoping, DTO boundary) so the ‘why’ survives staff changes.
- Generated API reference from Zod/OpenAPI; runbooks for the top incidents (DB down, auth failing, capture spam).

Team adoption strategy

- **Make the right thing the easy thing:** a feature scaffold generates the vertical slice above, so the pattern is the path of least resistance.
- **Enforce in CI, not in review:** lint rules and required checks catch drift automatically; reviews focus on design.

- **Definition of Done** includes tests for new invariants, docs/ADR when a decision changes, and a rollback note.
- **Lightweight RFCs** for cross-cutting changes; a visible tech-debt register so P1/P2 items are scheduled, not forgotten.
- Roll standards out incrementally — new code complies immediately, existing code is migrated as it is touched, never in a stop-the-world sweep.

AI usage declaration

Declaration

AI tooling was used to assist with brainstorming structure and drafting the prose of this report. All engineering judgements — the prioritisation of issues, the phased sequencing, the refactor design, and the standards themselves — reflect decisions I made and reviewed manually against the actual codebase. Every code example and architectural claim was checked against the implementation I built for Task A.